

# Lesson 5: Java Syntax Rules And Debugging Basics

Generated Jul 7, 2026 1:26 PM

---

## What We'll Learn

---

By the end of this lesson, we can:

- Identify common Java syntax errors.
- Correct missing semicolons, brackets, quotation marks, and wrong keywords.
- Explain why syntax rules matter.

## Words We'll Use

---

- Syntax - rules for writing valid code.
- Error - problem that prevents or affects execution.
- Semicolon - symbol that ends many Java statements.
- Brace - symbol used to group code.
- Debugging - finding and fixing errors.

## Before We Start

---

Why do sentences need punctuation? How is punctuation similar to syntax in programming?

# Let's Understand

## Java Syntax: Spot It, Fix It

### Missing semicolon

```
System.out.println("Hi")
```

### Correct

```
System.out.println("Hi");
```

### Wrong capitalization

```
system.out.println("Hi");
```

### Correct

```
System.out.println("Hi");
```

### Missing quotation mark

```
System.out.println("Hi);
```

### Correct

```
System.out.println("Hi");
```

**Checklist: capitalization, quotes, semicolon, braces.**

Let's work through Lesson 5: Java Syntax Rules And Debugging Basics together. Our focus is recognizing Java syntax rules and correcting beginner errors. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means:

- Java is a programming language we use to write instructions for a computer.
- A Java program is written as source code, saved in a `.java` file, compiled, then run.
- The JDK gives us the tools for building Java programs; the JVM runs the compiled bytecode.
- An IDE helps us write, organize, run, and debug code in one workspace.
- A beginner Java program usually has a class, a main method, statements, braces, and comments.
- We read code from the outside container toward the inside instructions, then we trace what will display.

How we study it step by step:

- First, we connect the lesson to a familiar situation: Why do sentences need punctuation? How is punctuation similar to syntax in programming?
- Next, we name the important parts so everyone uses the same vocabulary.
- Then, we study Semicolons and ask: what does it do, where do we see it, and how can we check it?
- Then, we study Quotation marks and ask: what does it do, where do we see it, and how can we check it?
- Then, we study Braces and ask: what does it do, where do we see it, and how can we check it?
- Then, we study Capitalization and ask: what does it do, where do we see it, and how can we check it?

- Then, we study Debugging and ask: what does it do, where do we see it, and how can we check it?
- After that, we compare a correct example with a common wrong version.
- We trace the example slowly before running, drawing, or submitting anything.
- Finally, we explain the result in our own words so the activity is not just copying.

Rules and patterns we should remember:

- Rule: Java is case-sensitive, so `System` and `system` are not the same. Example:  
`System.out.println("Hi");` works, but `system.out.println("Hi");` causes an error.  
 Check: Which word must start with a capital S?
- Rule: Text output must be inside double quotation marks. Example:  
`System.out.println("Hello, Java!");` displays Hello, Java! Check: What text will appear on the screen?
- Rule: Many Java statements end with a semicolon. Example: `int score = 90;` is complete, but `int score = 90` is missing its ending mark. Check: What symbol should we add at the end?
- Rule: Braces `{` and `}` must be paired because they show where a block starts and ends.  
 Example: `public class Demo { public static void main(String[] args) { } }` has matching class and main braces. Check: How many opening and closing braces can we count?
- Rule: Comments explain code for humans and are ignored when the program runs. Example: `// This line prints a greeting helps us understand the next statement.` Check: Will a comment display in the console?

Common mistakes we will watch for:

- Forgetting capitalization in Java keywords or class names.
- Missing quotation marks, semicolons, or closing braces.
- Thinking comments run as code.
- Changing code without predicting the output first.
- Submitting a screenshot without explaining what happened.

Mini-check before practice:

- What part of the Java program is the container?
- Where does the program start running?
- What line displays output?
- What syntax rule should we check before running?

When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

# Let's Practice

---

Let's practice with these activities:

Activity 1: Syntax Spot-The-Error Type: individual Time: 10 minutes Instruction: Find beginner Java syntax mistakes. Student task:

- Check capitalization.
- Check quotation marks.
- Check semicolons.
- Check braces.
- Circle or list each mistake.

Code / model:

```
public class SyntaxCheck {  
    public static void main(String[] args) {  
        system.out.println("Hi")  
        System.out.println("Java")  
    }  
}
```

Output: Submit the list of errors and corrected code.

Activity 2: Fix One Line At A Time Type: pair Time: 10 minutes Instruction: Correct each broken line and explain the rule. Student task:

- Fix `System.out.println("Hello").`
- Fix `system.out.println("Hi");.`
- Fix `System.out.println(Hi);.`
- Write the rule used for each fix.

Output: Submit three corrected lines and three matching rules.

Activity 3: Error Explanation Type: individual Time: 8 minutes Instruction: Explain why syntax matters. Student task:

- Choose one corrected error.
- Write what was wrong.
- Write how the correction helps Java understand the instruction.

Output: Submit a three-sentence explanation.

Activity 4: Create An Error For A Partner Type: pair Time: 10 minutes Instruction: Write one intentionally broken Java line for a partner to fix. Student task:

- Create one missing semicolon error or quote error.
- Trade with a partner.
- Fix your partner's line.
- Explain the correction.

Output: Submit the broken line, corrected line, and explanation.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Use the syntax checklist before leaving. Student task:

- Write one capitalization rule.
- Write one punctuation rule.
- Write one brace rule.

Output: Submit the three-rule checklist.

## Now We Apply

---

Now we apply it through these tasks:

1. Correct a broken Hello World program with at least five errors.
2. Make a personal Java Syntax Checklist.
3. Write three beginner syntax rules.

## Challenge Task

---

Debug a short Java program with at least eight errors. Submit corrected code and a correction log.

## How Our Work Will Be Checked

---

Criteria - 20 points total

- Correctness (5): Output or answer follows the required concept and instructions.
- Completeness (4): All required parts are present.
- Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order.
- Explanation (3): Student can explain what the work does and why it is correct.
- Neatness/readability (2): Work is easy to read, label, and check.
- Effort and revision (2): Student improves the work after feedback.

## Let's Reflect

---

Let's reflect:

- What concept from this lesson is clearest to you now?
- What mistake did you notice or correct during practice?
- Which activity helped you understand the lesson best?
- How can this skill help you design or write a Java program?
- What do you still need to practice before the next lesson?

## How We Show Learning

---

Evidence we will use to check learning:

- Common syntax errors are found and corrected.
- Correction log explains each fix.
- Corrected code runs or is logically valid.
- Completed guided-practice work with visible corrections or annotations.
- A short explanation connecting the output to the lesson target.

## Student Activities

---

Syntax Spot-the-Error: circle mistakes in five samples. Fix the Error Race: correct semicolons and braces. Capitalization Check. Quotation Mark Repair. Error Explanation: write why each correction is needed. Create an Error: write one wrong line for a partner to fix. Correct a broken Hello World program with at least five errors. Make a personal Java Syntax Checklist. Write three beginner syntax rules.